BACS2063 Data Structures and Algorithms

ASSIGNMENT 202501

Declaration

- I confirm that I have read and complied with all the terms and conditions of Tunku Abdul Rahman University of Management and Technology's plagiarism policy.
- I declare that this assignment is free from all forms of plagiarism and for all intents and purposes is my own properly derived work.

Student Name	Student ID	Programme & Tut.Group	Signature
CHANG JIA JIN	24WMR01122	RSD2S1G1	Chang
LUM GANG LI	24WMR01140	RSD2S1G1	GARY
SENG YUAN JUN	24WMR01142	RSD2S1G1	JUN

Note: The submission date and time will be according to the timestamp recorded in Google Classroom for Assignment Submission.

Leader:Chang Jia Jin

Name of 2

Name of 3

Table of Contents

Part A: TEAM REPORT **3**
1. Abstract Data Type (ADT) 3
1.1 Linked List ADT Specification 3
1.2 ADT Implementation 6
a. Interface class (LinkedList.java) 6
b. Implementation class (LinkedListInterface.java) 6
Part B. INDIVIDUAL REPORT **10**
2. Use of ADTs 10
CHANG JIA JIN 10
1. Source Code (SearchJobControl.java) 10
2. Screenshots 15
LUM GANG LI 18
1. Source codes (ApplicantManagementControl.java) 18
2. Screenshots 24
SENG YUAN JUN 31
1. Source codes (JobManage.java) 31
2. Screenshots 39

Part A: TEAM REPORT

1. Abstract Data Type (ADT)

1.1 Linked List ADT Specification

- A linked list of common elements LinkedList<T> supports dynamic insertion, deletion, replacement, retrieval, and inspection of elements by position or value.

Header	boolean add(T newEntry)
Description	Add a new element to the set.
Pre-condition	NewEntry must not be null, and should not exist in the set.
Post-condition	The current set has been updated.
Return	If the newEntry was not already in the set, then it is added and the method returns true. Otherwise the set remains unchanged, and the method returns false.

Header	Boolean add(int newPosition, T newEntry)
Description	Insert a new element at position newPosition.
Pre-condition	newPosition more or equal than 1 and less or equal than number of entry plus 1; newEntry must not be null.
Post-condition	newEntry occupies index newPosition; subsequent element shifts right; numberOfEntries increase 1.
Return	If insertion successfully returns true. If the newPosition out of range returns false.

Header	T remove(int givenPosition)
Description	Remove and return the element at the given position.
Pre-condition	Given position more or equal than 1 and less or equal than number or entry.
Post-condition	Element at givenPosition is removed; subsequent elements shift left; numberOfEntries decrease 1.
Return	Return the removed entry or null if operation fails.

Header	boolean replace(int givenPosition, T newEntry)
Description	Replace the element at givenPosition with newEntry.
Pre-condition	givenPosition more or equal than 1 and less or equal than numberOfEntries; newEntry must not be null.
Post-condition	Element at givenPosition is set to newEntry.
Return	If replacement is successful, return true. Otherwise return false.

Header	T getEntry(int givenPosition)
Description	Retrieve but do not remove the element at givenPosition
Pre-condition	givenPosition more or equal than 1 and less or equal than numberOfEntries.
Post-condition	List unchanged.
Return	Return the element at givenPosition, or null if out of range.

Header	boolean contains(T anEntry)
Description	Check whether anEntry is present anywhere in the list.
Pre-condition	anEntry must not be null.
Post-condition	List unchanged.
Return	If an equal element is found, return true. Otherwise return false.

Header	int getNumberOfEntries()
Description	Return the current number of elements in the list.
Pre-condition	-
Post-condition	List unchanged.
Return	Return the integer size of the list.

Header	boolean isEmpty()
Description	Check whether the list contains no elements.
Pre-condition	-
Post-condition	List unchanged.
Return	If numberOfEntres = 0, return true. Otherwise return false.

Header	boolean isFull()
Description	Check whether the list cannot accept more elements.
Pre-condition	-
Post-condition	List unchanged.
Return	Always returns false.

Header	void clear()
Description	Remove all elements from the list.
Pre-condition	-
Post-condition	List is empty; size reset to 0.
Return	-

1.2 ADT Implementation

a. Interface class (LinkedList.java)

```
package adt;

public interface LinkedListInterface<T> {
    public boolean add(T newEntry);
    public boolean add(int newPosition, T newEntry);
    public T remove(int givenPosition);
    public boolean replace(int givenPosition, T newEntry);
    public T getEntry(int givenPosition);
    public boolean contains(T anEntry);
    public int getNumberOfEntries();
    public boolean isEmpty();
    public boolean isFull();

}
```

b. Implementation class (LinkedListInterface.java)

```
package adt;

public class LinkedList<T> implements LinkedListInterface<T> {

    private Node firstNode;
    private int numberOfEntries;

    public LinkedList() {
        clear();
    }

    @Override
    public boolean add(T newEntry) {
        Node newNode = new Node(newEntry);
        if (isEmpty()) {
            firstNode = newNode;
        } else {
            Node currentNode = firstNode;
            while (currentNode.next != null) {
                currentNode = currentNode.next;
            }
            currentNode.next = newNode;
        }
        numberOfEntries++;
        return true;
    }
```

```
@Override
public boolean add(int newPosition, T newEntry) {
    if (newPosition < 1 || newPosition > numberOfEntries + 1) {
        return false;
    }
    Node newNode = new Node(newEntry);
    if (isEmpty() || newPosition == 1) {
        newNode.next = firstNode;
        firstNode = newNode;
    } else {
        Node nodeBefore = firstNode;
        for (int i = 1; i < newPosition - 1; i++) {
            nodeBefore = nodeBefore.next;
        }
        newNode.next = nodeBefore.next;
        nodeBefore.next = newNode;
    }
    numberOfEntries++;
    return true;
}

@Override
public T remove(int givenPosition) {
    T result = null;
    if (givenPosition >= 1 && givenPosition <= numberOfEntries) {
        if (givenPosition == 1) {
            result = firstNode.data;
            firstNode = firstNode.next;
        } else {
            Node nodeBefore = firstNode;
            for (int i = 1; i < givenPosition - 1; i++) {
                nodeBefore = nodeBefore.next;
            }
            result = nodeBefore.next.data;
            nodeBefore.next = nodeBefore.next.next;
        }
        numberOfEntries--;
    }
    return result;
}

@Override
public boolean replace(int givenPosition, T newEntry) {
    if (givenPosition < 1 || givenPosition > numberOfEntries) {
        return false;
    }
    Node currentNode = firstNode;
    for (int i = 1; i < givenPosition; i++) {
```

```
        currentNode = currentNode.next;
    }
    currentNode.data = newEntry;
    return true;
}

@Override
public T getEntry(int givenPosition) {
    if (givenPosition < 1 || givenPosition > numberOfEntries) {
        return null;
    }
    Node currentNode = firstNode;
    for (int i = 1; i < givenPosition; i++) {
        currentNode = currentNode.next;
    }
    return currentNode.data;
}

@Override
public boolean contains(T anEntry) {
    Node currentNode = firstNode;
    while (currentNode != null) {
        if (anEntry.equals(currentNode.data)) {
            return true;
        }
        currentNode = currentNode.next;
    }
    return false;
}

@Override
public int getNumberOfEntries() {
    return numberOfEntries;
}

@Override
public boolean isEmpty() {
    return numberOfEntries == 0;
}
```

```
@Override
  public boolean isFull() {
     return false;
  }

  private class Node {
     private T data;
     private Node next;

     private Node(T data) {
        this(data, null);
     }

     private Node(T data, Node next) {
        this.data = data;
        this.next = next;
     }
  }
}
```

Part B. INDIVIDUAL REPORT

2. Use of ADTs

Name	Student ID	Prog / Tut.Grp	Signature
CHANG JIA JIN	24WMR01122	RSD2 S1 G1	Chang

Module name: Search Module

1. Source Code (SearchJobControl.java)

```
package control;
/**
 *
 * @author CHANG JIA JIN
 */
import adt.LinkedListInterface;
import adt.SortedArrayList;
import boundary.SearchJobUI;
import entity.Job;
import entity.Search;
import entity.SearchResult;
import entity.SearchEntry;

import java.util.Scanner;

public class SearchJobControl {

    public Search job;
    private final SortedArrayList<Search> searchList;
    private final SearchEntry[] trackedJobs = new SearchEntry[100];
    private int trackedCount = 0;
    private final LinkedListInterface<Job> jobList;


    public SearchJobControl(SortedArrayList<Search> searchList,LinkedListInterface<Job> jobList) {
        this.searchList = searchList;
        this.jobList= jobList;

        }
    }

    public void run() {
        Scanner scanner = new Scanner(System.in);
        SearchJobUI ui = new SearchJobUI(this);


        while (true) {
            ui.displaySearchMenu();
            String choice = scanner.nextLine().trim();

            if (choice.equals("0")) {
                System.out.println("Exiting search...");
```

```
            break;
        }

        MenuChoice(choice, scanner, ui);
    }

    new MainMenu().run();
}

public boolean MenuChoice(String choice, Scanner scanner, SearchJobUI ui) {
    switch (choice) {
        case "1" -> {
            Search[] results = searchByKeyword(scanner, ui);
            ui.displayJobs(results);
            return true;
        }
        case "2" -> {
            ui.displayJobs(searchList.getAllEntries());
            return true;
        }
        case "3" -> {
            displaySearchReport(generateSearchReportJobs(), this);
            return true;
        }
        case "0" -> {
            MainMenu mm = new MainMenu();
            mm.run();
            return true;
        }
        default -> {
            System.out.println("Invalid choice.");
            return true;
        }
    }
}

private Search[] searchByKeyword(Scanner scanner, SearchJobUI ui) {
    String keyword = ui.searchKeyword(scanner);
    if (keyword.equals("0")) {
        System.out.println("Back to menu.");
        return new Search[0];
    }
    return searchJobs(keyword);
}

public Search[] searchJobs(String keyword) {
    SearchResult[] matches = new SearchResult[jobList.getNumberOfEntries()];
    int count = 0;
    String k = keyword.toLowerCase();

    for (Search job : searchList.getAllEntries()) {
        int rank = calculateRelevanceRank(job, k);
        if (rank > 0) {
            matches[count++] = new SearchResult(job, rank);
            updateSearchFrequency(job);
        }
    }
```

```
    Search[] output = new Search[count];
    for (int i = 0; i < count; i++) {
        output[i] = matches[i].getJob();
    }
    return output;
}

private int calculateRelevanceRank(Search job, String keyword) {
    int rank = 0;

    if (job.getTitle().toLowerCase().contains(keyword)) {
        rank += 3;
    }
    if (job.getDescription().toLowerCase().contains(keyword)) {
        rank += 2;
    }
    if (job.getCompanyName().toLowerCase().contains(keyword)) {
        rank += 1;
    }
    if (job.getLocation().toLowerCase().contains(keyword)) {
        rank += 1;
    }
    if (job.getJobType().toLowerCase().contains(keyword)) {
        rank += 1;
    }
    if (String.valueOf(job.getSalary()).contains(keyword)) {
        rank += 1;
    }

    if (rank == 0) {
        if (searchList.fuzzySearch(keyword, job.getTitle().toLowerCase()) <= 5) {
            rank += 3;
        } else if (searchList.fuzzySearch(keyword, job.getDescription().toLowerCase()) <= 2) {
            rank += 2;
        } else if (searchList.fuzzySearch(keyword, job.getCompanyName().toLowerCase()) <= 2) {
            rank += 1;
        } else if (searchList.fuzzySearch(keyword, job.getLocation().toLowerCase()) <= 2) {
            rank += 1;
        } else if (searchList.fuzzySearch(keyword, job.getJobType().toLowerCase()) <= 2) {
            rank += 1;
        } else if (searchList.fuzzySearch(keyword, String.valueOf(job.getSalary())) <= 2) {
            rank += 1;
        }
    }

    return rank;
}


private void updateSearchFrequency(Search job) {
    for (int i = 0; i < trackedCount; i++) {
        if (trackedJobs[i].job.equals(job)) {
            trackedJobs[i].count++;
            return;
        }
    }
    trackedJobs[trackedCount++] = new SearchEntry(job, 1);
}
```

```
    public Search[] generateSearchReportJobs() {
        Search[] report = new Search[trackedCount];
        int[] frequency = new int[trackedCount];

        for (int i = 0; i < trackedCount; i++) {
            report[i] = trackedJobs[i].job;
            frequency[i] = trackedJobs[i].count;
        }

        for (int i = 0; i < trackedCount - 1; i++) {
            for (int j = 0; j < trackedCount - i - 1; j++) {
                if (frequency[j] < frequency[j + 1]) {
                    int tempFreq = frequency[j];
                    frequency[j] = frequency[j + 1];
                    frequency[j + 1] = tempFreq;

                    Search tempJob = report[j];
                    report[j] = report[j + 1];
                    report[j + 1] = tempJob;
                }
            }
        }

        return report;
    }

    public int getSearchFrequency(Search job) {
        for (int i = 0; i < trackedCount; i++) {
            if (trackedJobs[i].job.equals(job)) {
                return trackedJobs[i].count;
            }
        }
        return 0;
    }

    public void displaySearchReport(Search[] jobs, SearchJobControl control) {
    System.out.println();
    System.out.println("                                Job Search Report");
    System.out.println("-----------------------------------------------------------------------------------------------");
    System.out.printf("| %-20s | %-15s | %-10s | %-10s | %-13s |\n", "Title", "Company", "Type", "Salary",
"Search Count");
    System.out.println("-----------------------------------------------------------------------------------------------");

    if (jobs == null || jobs.length == 0) {
        System.out.printf("| %-83s |\n", "No jobs have been searched yet.");
    } else {
        for (Search job : jobs) {
            if (job == null) continue;
            int count = control.getSearchFrequency(job);
            System.out.printf("| %-20s | %-15s | %-10s | RM %-7.2f | %-13d |\n",
                    job.getTitle(),
                    job.getCompanyName(),
                    job.getJobType(),
                    job.getSalary(),
                    count);
        }
    }
```

```
    System.out.println("----------------------------------------------------------------------------------------------------");
}

}
```

2. Screenshots

Includes screenshot of user interface input and outputs for core functions of your modules. Label your screenshots clearly.

1.

```
=========== Internship Applicaiton ==========
1. Job Management
2. Applicant Management
3. Search
0. Exit system
========================================
Enter Choice: 3
```

When in the Main Menu page, we will enter 3 to go through the SearchModule page

2.

```
===== Job Search Menu =====
1. Search by keyword
2. Show All Jobs
3. View Job Search Report
0. Exit
Enter choice: 1
```

In the search module page it will display 4 option in the search menu as well, we can center choice 1 to search the job by keyword

3.

```
===== Job Search Menu =====
1. Search by keyword
2. Show All Jobs
3. View Job Search Report
0. Exit
Enter choice: 1
Enter keyword to search (or 0 to back): software desceslop
```

After enter choice 1 the system will let us to enter the keyword, the keyword can be the exactly search , fuzzy search or keyword search

Exact Search:

```
===== Job Search Menu =====
1. Search by keyword
2. Show All Jobs
3. View Job Search Report
0. Exit
Enter choice: 1
Enter keyword to search (or 0 to back): software developer

===== Search Results (1) =====

------------------------------
          Job #1
------------------------------
Title       : Software Developer
Description : Develop and maintain Java applications.
Company     : TechSoft
Location    : Kuala Lumpur
Job Type    : Full-Time
Salary      : 1500.0
------------------------------
```

Fuzzy Search:

```
===== Job Search Menu =====
1. Search by keyword
2. Show All Jobs
3. View Job Search Report
0. Exit
Enter choice: 1
Enter keyword to search (or 0 to back): software desceslop

===== Search Results (1) =====

------------------------------
          Job #1
------------------------------
Title       : Software Developer
Description : Develop and maintain Java applications.
Company     : TechSoft
Location    : Kuala Lumpur
Job Type    : Full-Time
Salary      : 1500.0
------------------------------
```

Keyword Search:

```
===== Job Search Menu =====
1. Search by keyword
2. Show All Jobs
3. View Job Search Report
0. Exit
Enter choice: 1
Enter keyword to search (or 0 to back): software

===== Search Results (1) =====

------------------------------
          Job #1
------------------------------
Title       : Software Developer
Description : Develop and maintain Java applications.
Company     : TechSoft
Location    : Kuala Lumpur
Job Type    : Full-Time
Salary      : 1500.0
------------------------------
```

4.

```
===== Job Search Menu =====
1. Search by keyword
2. Show All Jobs
3. View Job Search Report
0. Exit
Enter choice: 2
```

Once the search results came out ,it will automatically show the menu below the search result, now we can enter 2 to see all the available job as well

```
===== Job Search Menu =====
1. Search by keyword
2. Show All Jobs
3. View Job Search Report
0. Exit
Enter choice: 2

===== Search Results (5) =====

------------------------------
          Job #1
------------------------------
Title       : App Developer
Description : Develop and maintain mobile applications.
Company     : MobileApp
Location    : Penang
Job Type    : Full-Time
Salary      : 1000.0
------------------------------

------------------------------
          Job #2
------------------------------
Title       : Database Administrator
Description : Manage and maintain databases.
Company     : DBSolutions
Location    : Penang
Job Type    : Full-Time
Salary      : 1600.0
------------------------------

------------------------------
          Job #3
------------------------------
Title       : Software Developer
Description : Develop and maintain Java applications.
Company     : TechSoft
Location    : Kuala Lumpur
Job Type    : Full-Time
Salary      : 1500.0
------------------------------

------------------------------
          Job #4
------------------------------
Title       : System Analyst
Description : Analyze and design systems for clients.
Company     : SysTech
Location    : Kuala Lumpur
Job Type    : Part-Time
Salary      : 1300.0
------------------------------

------------------------------
          Job #5
------------------------------
Title       : Web Developer
Description : Develop and maintain websites.
Company     : WebTech
Location    : Johor Bahru
Job Type    : Full-Time
Salary      : 1200.0
------------------------------
```

These are all the available job in the system, all the job are arranged in a sequence way(A-Z)

5.

```
===== Job Search Menu =====
1. Search by keyword
2. Show All Jobs
3. View Job Search Report
0. Exit
Enter choice: 3
```

Go back to the menu page again , we can enter 3 to see how many of the job have been searched, except the show all job

```
                    Job Search Report
-------------------------------------------------------------------------------------------
| Title                | Company          | Type        | Salary      | Search Count  |
-------------------------------------------------------------------------------------------
| Software Developer   | TechSoft         | Full-Time   | RM 1500.00  | 4             |
-------------------------------------------------------------------------------------------
```

This is the report that had mention that how many of the job have been searched

Name	Student ID	Prog / Tut.Grp	Signature
LUM GANG LI	24WMR01140	RSD2 S1G1	Gary

Module name: Applicant Management (Job Seeker) Module

1. Source codes (ApplicantManagementControl.java)

```
package control;
/**
 *
 * @author LUM GANG LI
 */
import adt.LinkedList;
import adt.LinkedListInterface;
import boundary.ApplicantManagementUI;
import entity.Applicant;
import utility.MessageUI;

import java.util.Scanner;
import java.util.InputMismatchException;

public class ApplicantManagementControl {

    private final LinkedListInterface<Applicant> applicants;
    private final ApplicantManagementUI ui;
    private final MessageUI messageUI;
    private final Scanner scanner;
    private int nextApplicantNumber;

    public ApplicantManagementControl(boolean loadSampleData) {
        applicants = new LinkedList<>();
        ui         = new ApplicantManagementUI();
        messageUI  = new MessageUI();
        scanner    = new Scanner(System.in);
        nextApplicantNumber = 0;

        if (loadSampleData) {
            initializeSampleData();
            nextApplicantNumber = applicants.getNumberOfEntries();
        }
    }

    public void run() {
        boolean quit = false;
        while (!quit) {
```

```
        ui.displayMenu();
        int choice = getNextInt();
        switch (choice) {
            case 1 ->
                listAllApplicants();
            case 2 ->
                addApplicant();
            case 3 ->
                searchByLocation();
            case 4 ->
                searchBySkill();
            case 5 ->
                updateApplicant();
            case 6 ->
                removeApplicant();
            case 7 ->
                generateReport();
            case 0 -> {
                messageUI.displayExitMessage();
                quit = true;
            }
            default ->
                messageUI.displayInvalidChoiceMessage();
        }
    }
}

private void listAllApplicants() {
    if (applicants.isEmpty()) {
        System.out.println("No applicants available.");
    } else {
        for (int i = 1; i <= applicants.getNumberOfEntries(); i++) {
            ui.displayApplicant(applicants.getEntry(i), i);
        }
    }
}

private void addApplicant() {
    System.out.println("\n=== Add New Applicant ===");
    Applicant newApp = getApplicantDetails();
    applicants.add(newApp);
    System.out.println("Applicant added successfully!");
}

   private Applicant getApplicantDetails() {
    String id = generateApplicantId();
    System.out.println("Assigned Applicant ID: " + id);

    System.out.print("Enter Name (letters only): ");
    String name = inputValidated("[A-Za-z ]+", "Invalid name.", "Enter Name (letters only): ");

    System.out.print("Enter Email: ");
    String email = inputValidated(
            "^[^@\\s]+@[^@\\s]+\\.[^@\\s]+$", "Invalid email format.", "Enter Email: ");

    System.out.print("Enter Location (letters only): ");
    String location = inputValidated("[A-Za-z ]+", "Invalid location.", "Enter Location: ");
```

```
    var skillsList = new LinkedList<String>();
    System.out.print("How many skills? ");
    int skillCount = getNextInt();
    for (int i = 1; i <= skillCount; i++) {
        System.out.print("Enter Skill " + i + ": ");
        String skill = inputValidated("[A-Za-z ]+", "Invalid skill.", "Enter Skill: ");
        skillsList.add(skill);
    }

   LinkedListInterface<String> jobTypesList = new LinkedList<>();
    int typeChoice;
    do {
        System.out.println("Select Desired Job Type:");
        System.out.println("1. Fulltime");
        System.out.println("2. Parttime");
        System.out.print("Enter choice (1 or 2): ");
        typeChoice = getNextInt();
        if (typeChoice != 1 && typeChoice != 2) {
            messageUI.displayInvalidChoiceMessage();
        }
    } while (typeChoice != 1 && typeChoice != 2);

    if (typeChoice == 1) {
        jobTypesList.add("Fulltime");
    } else {
        jobTypesList.add("Parttime");
    }


    return new Applicant(id, name, email, location, skillsList, jobTypesList);
}

private void searchByLocation() {
    System.out.print("Enter Location to search: ");
    String loc = inputValidated("[A-Za-z ]+", "Invalid location.", "Enter Location: ");
    var results = filterByLocation(loc);
    displaySearchResults(results, "Applicants in " + loc);
}

private void searchBySkill() {
    System.out.print("Enter Skill to search: ");
    String skill = inputValidated("[A-Za-z ]+", "Invalid skill.", "Enter Skill: ");
    var results = filterBySkill(skill);
    displaySearchResults(results, "Applicants with skill: " + skill);
}

private void displaySearchResults(
        LinkedListInterface<Applicant> results,
        String title
) {
    System.out.println("\n=== " + title + " ===");
    if (results.isEmpty()) {
        System.out.println("No applicants found.");
    } else {
        for (int i = 1; i <= results.getNumberOfEntries(); i++) {
            ui.displayApplicant(results.getEntry(i), i);
        }
    }
```

```
}

private void updateApplicant() {
    System.out.print("Enter Applicant ID to update (A###): ");
    String id = inputApplicantId();
    Applicant a = getApplicant(id);
    if (a == null) {
        System.out.println("Applicant not found!");
        return;
    }
    ui.displayApplicant(a, 0);

    boolean editing = true;
    while (editing) {
        ui.displayUpdateOptions();
        int opt = getNextInt();
        switch (opt) {
            case 1 -> {
                System.out.print("New Name: ");
                a.setName(inputValidated("[A-Za-z ]+", "Invalid name.", "Enter Name: "));
            }
            case 2 -> {
                System.out.print("New Email: ");
                a.setEmail(inputValidated(
                        "^[^@\\s]+@[^@\\s]+\\.[^@\\s]+$", "Invalid email.", "Enter Email: "));
            }
            case 3 -> {
                System.out.print("New Location: ");
                a.setLocation(inputValidated(
                        "[A-Za-z ]+", "Invalid location.", "Enter Location: "));
            }
            case 4 -> {
                var newSkills = new LinkedList<String>();
                System.out.print("How many skills? ");
                int cnt = getNextInt();
                for (int i = 1; i <= cnt; i++) {
                    System.out.print("Skill " + i + ": ");
                    String s = inputValidated(
                            "[A-Za-z ]+", "Invalid skill.", "Enter Skill: ");
                    newSkills.add(s);
                }
                a.setSkillsList(newSkills);
            }
            case 5 -> {
                var newJobTypes = new LinkedList<String>();
                int jtChoice;
                do {
                    System.out.println("Select Desired Job Type:");
                    System.out.println("1. Fulltime");
                    System.out.println("2. Parttime");
                    System.out.print("Enter choice (1 or 2): ");
                    jtChoice = getNextInt();
                    if (jtChoice != 1 && jtChoice != 2) {
                        messageUI.displayInvalidChoiceMessage();
                    }
                } while (jtChoice != 1 && jtChoice != 2);

                if (jtChoice == 1) {
```

```
                newJobTypes.add("Fulltime");
            } else {
                newJobTypes.add("Parttime");
            }

            a.setDesiredJobTypesList(newJobTypes);
        }

        case 0 ->
            editing = false;
        default ->
            messageUI.displayInvalidChoiceMessage();
    }
}
System.out.println("Applicant updated successfully!");
}

private void removeApplicant() {
    System.out.print("Enter Applicant ID to remove: ");
    String id = inputApplicantId();
    Applicant a = getApplicant(id);
    if (a == null) {
        System.out.println("Applicant not found.");
        return;
    }
    System.out.print("Confirm remove \"" + id + "\"? (Y/N): ");
    if (scanner.nextLine().equalsIgnoreCase("Y")) {
        for (int i = 1; i <= applicants.getNumberOfEntries(); i++) {
            if (applicants.getEntry(i).getApplicantId().equalsIgnoreCase(id)) {
                applicants.remove(i);
                System.out.println("Removed: " + id);
                renumberApplicants();
                return;
            }
        }
    } else {
        System.out.println("Removal cancelled.");
    }
}

private String inputApplicantId() {
    String id;
    while (true) {
        id = scanner.nextLine().trim();
        if (id.matches("A\\d{3}")) return id;
        System.out.print("Invalid ID. Re-enter (A###): ");
    }
}

private Applicant getApplicant(String id) {
    for (int i = 1; i <= applicants.getNumberOfEntries(); i++) {
        if (applicants.getEntry(i).getApplicantId().equalsIgnoreCase(id)) {
            return applicants.getEntry(i);
        }
    }
    return null;
}
```

```
private void renumberApplicants() {
    for (int i = 1; i <= applicants.getNumberOfEntries(); i++) {
        applicants.getEntry(i).setApplicantId(String.format("A%03d", i));
    }
    nextApplicantNumber = applicants.getNumberOfEntries();
}

private LinkedListInterface<Applicant> filterByLocation(String loc) {
    LinkedListInterface<Applicant> res = new LinkedList<>();
    for (int i = 1; i <= applicants.getNumberOfEntries(); i++) {
        if (applicants.getEntry(i).getLocation().equalsIgnoreCase(loc)) {
            res.add(applicants.getEntry(i));
        }
    }
    return res;
}

private LinkedListInterface<Applicant> filterBySkill(String skill) {
    LinkedListInterface<Applicant> res = new LinkedList<>();
    for (int i = 1; i <= applicants.getNumberOfEntries(); i++) {
        for (String s : applicants.getEntry(i).getSkills()) {
            if (s.equalsIgnoreCase(skill)) {
                res.add(applicants.getEntry(i));
                break;
            }
        }
    }
    return res;
}

private void initializeSampleData() {
    applicants.add(new Applicant("A001", "John Doe", "john.doe@example.com",
        "Kuala Lumpur", new String[]{"Java","SQL"}, new String[]{"Fulltime"}));
    applicants.add(new Applicant("A002", "Jane Smith", "jane.smith@example.com",
        "Penang", new String[]{"Python","Django"}, new String[]{"Parttime"}));
}

private String generateApplicantId() {
    nextApplicantNumber++;
    return String.format("A%03d", nextApplicantNumber);
}

private int getNextInt() {
    while (true) {
        try {
            int i = scanner.nextInt();
            scanner.nextLine();
            return i;
        } catch (InputMismatchException e) {
            messageUI.displayInvalidChoiceMessage();
            scanner.nextLine();
        }
    }
}

private void generateReport() {
    boolean back = false;
    while (!back) {
```

```
            ui.displayReportMenu();
            int choice = getNextInt();
            switch (choice) {
                case 1 -> reportAllApplicants();
                case 2 -> reportByLocation();
                case 3 -> reportBySkill();
                case 0 -> back = true;
                default -> messageUI.displayInvalidChoiceMessage();
            }
        }
    }

    private void reportAllApplicants() {
        ui.displayReportHeader("Applicant Summary Report");
        for (int i = 1; i <= applicants.getNumberOfEntries(); i++) {
            ui.displayReportRow(applicants.getEntry(i));
        }
        ui.displayReportFooter();
    }

    private void reportByLocation() {
        System.out.print("Enter location: ");
        String loc = scanner.nextLine();
        var res = filterByLocation(loc);
        ui.displayReportHeader("Applicants in \"" + loc + "\"");
        if (res.isEmpty()) {
            System.out.println("|                    No applicants found                    |");
        } else {
            for (int i = 1; i <= res.getNumberOfEntries(); i++) {
                ui.displayReportRow(res.getEntry(i));
            }
        }
        ui.displayReportFooter();
    }

    private void reportBySkill() {
        System.out.print("Enter skill: ");
        String skill = scanner.nextLine();
        var res = filterBySkill(skill);
        ui.displayReportHeader("Applicants with skill \"" + skill + "\"");
        if (res.isEmpty()) {
            System.out.println("|                    No applicants found                    |");
        } else {
            for (int i = 1; i <= res.getNumberOfEntries(); i++) {
                ui.displayReportRow(res.getEntry(i));
            }
        }
        ui.displayReportFooter();
    }
}
```

2. Screenshots

```
=========== Internship Applicaiton ===========
1. Job Management
2. Applicant Management
3. Search
0. Exit system
========================================
Enter Choice: 2
```

Choose 2 to enter applicant management ui

```
=========== Applicant Management ===========
1. List All Applicants
2. Add Applicant
3. Search by Location
4. Search by Skill
5. Update Applicant
6. Remove Applicant
7. Generate Report
0. Exit
Enter Choice: 1

================ Applicant 1 ================
ID       : A001
Name     : John Doe
Email    : john.doe@example.com
Location : Kuala Lumpur
Skills   : Java, SQL
JobTypes : Full-time
=============================================

================ Applicant 2 ================
ID       : A002
Name     : Jane Smith
Email    : jane.smith@example.com
Location : Penang
Skills   : Python, Django
JobTypes : Part-time
=============================================
```

Enter 1 will list all of the applicant (now this is the preset sample)

```
=========== Applicant Management ===========
1. List All Applicants
2. Add Applicant
3. Search by Location
4. Search by Skill
5. Update Applicant
6. Remove Applicant
7. Generate Report
0. Exit
Enter Choice: 2

=== Add New Applicant ===
Assigned Applicant ID: A003
Enter Name (letters only): Lee
Enter Email: Lee@gmail.com
Enter Location (letters only): johor
How many skills? 1
Enter Skill 1: java
Select Desired Job Type:
1. Fulltime
2. Parttime
Enter choice (1 or 2): 1
Applicant added successfully!
```

Enter 2 to allow user to type the applicant detail for adding new applicant

```
=========== Applicant Management ===========
1. List All Applicants
2. Add Applicant
3. Search by Location
4. Search by Skill
5. Update Applicant
6. Remove Applicant
7. Generate Report
0. Exit
Enter Choice: 3
Enter Location to search: johor

=== Applicants in johor ===

================ Applicant 1 ================
ID       : A003
Name     : Lee
Email    : Lee@gmail.com
Location : johor
Skills   : java
JobTypes : Fulltime
=============================================
```

Enter 3 For searching location and display the applicant based on search location

```
=========== Applicant Management ===========
1. List All Applicants
2. Add Applicant
3. Search by Location
4. Search by Skill
5. Update Applicant
6. Remove Applicant
7. Generate Report
0. Exit
Enter Choice: 4
Enter Skill to search: Java

=== Applicants with skill: Java ===

================ Applicant 1 ================
ID       : A001
Name     : John Doe
Email    : john.doe@example.com
Location : Kuala Lumpur
Skills   : Java, SQL
JobTypes : Fulltime
=============================================

================ Applicant 2 ================
ID       : A003
Name     : Lee
Email    : Lee@gmail.com
Location : johor
Skills   : java
JobTypes : Fulltime
=============================================
```

Enter 4 for searching skill and display the applicant based on skill searched

```
=========== Applicant Management ===========
1. List All Applicants
2. Add Applicant
3. Search by Location
4. Search by Skill
5. Update Applicant
6. Remove Applicant
7. Generate Report
0. Exit
Enter Choice: 5
Enter Applicant ID to update (A###): A001

================ Applicant 0 ================
ID       : A001
Name     : John Doe
Email    : john.doe@example.com
Location : Kuala Lumpur
Skills   : Java, SQL
JobTypes : Fulltime
=============================================

=========== Update Applicant ===========
1. Name
2. Email
3. Location
4. Skills
5. Desired Job Types
0. Cancel
Enter choice: 5
Select Desired Job Type:
1. Fulltime
2. Parttime
Enter choice (1 or 2): 2

=========== Update Applicant ===========
1. Name
2. Email
3. Location
4. Skills
5. Desired Job Types
0. Cancel
Enter choice: 0
Applicant updated successfully!
```

Enter 5 to allow user change the detail of applicant in every category and update successfully will be display and to check just go back to option 1 (list all applicant)

```
=========== Applicant Management ===========
1. List All Applicants
2. Add Applicant
3. Search by Location
4. Search by Skill
5. Update Applicant
6. Remove Applicant
7. Generate Report
0. Exit
Enter Choice: 6
Enter Applicant ID to remove: A003
Confirm remove "A003"? (Y/N): y
Removed: A003
```

Enter 6 can remove any applicant by searching the Applicant ID and let user to make confirmation

```
=========== Applicant Management ===========
1. List All Applicants
2. Add Applicant
3. Search by Location
4. Search by Skill
5. Update Applicant
6. Remove Applicant
7. Generate Report
0. Exit
Enter Choice: 7

======== Applicant Report Generation ========
1. Report All Applicants
2. Report by Location
3. Report by Skill
0. Back to Menu
Enter choice:
```

Enter 7 to enter report menu

```
======== Applicant Report Generation ========
1. Report All Applicants
2. Report by Location
3. Report by Skill
0. Back to Menu
Enter choice: 1

| Applicant Summary Report |
-------------------------------------------------------------------
| ID   | Name           | Email                | Location | Skills | Job Types |
-------------------------------------------------------------------
| A001 | John Doe       | john.doe@example.com | Kuala Lumpur | Java, SQL | Parttime  |
| A002 | Jane Smith     | jane.smith@example.com | Penang   | Python, Django | Parttime  |
-------------------------------------------------------------------
```

from here enter 1 is list all of the applicant in a report format

```
======== Applicant Report Generation ========
1. Report All Applicants
2. Report by Location
3. Report by Skill
0. Back to Menu
Enter choice: 2
Enter location: penang

| Applicants in "penang" |
-------------------------------------------------------------------
| ID   | Name           | Email                | Location | Skills | Job Types |
-------------------------------------------------------------------
| A002 | Jane Smith     | jane.smith@example.com | Penang   | Python, Django | Parttime  |
-------------------------------------------------------------------
```

Enter 2 can generate report based on location and list all of the applicant based on location searched in a report format

```
======== Applicant Report Generation ========
1. Report All Applicants
2. Report by Location
3. Report by Skill
0. Back to Menu
Enter choice: 3
Enter skill: java

| Applicants with skill "java" |
-------------------------------------------------------------------
| ID   | Name           | Email                | Location | Skills | Job Types |
-------------------------------------------------------------------
| A001 | John Doe       | john.doe@example.com | Kuala Lumpur | Java, SQL | Parttime  |
-------------------------------------------------------------------
```

Enter 3 can generate report based on skill and the applicant based on skill searched in a report format

Name	Student ID	Prog / Tut.Grp	Signature
SENG YUAN JUN	24WPF01142	RSD2 S1G1	JUN

Module name: Job Management (Employee) Module

1. Source codes (JobManage.java)

```
package control;
/**
 *
 * @author Yuan Jun
 */

import adt.LinkedList;
import adt.LinkedListInterface;
import boundary.JobManagementUI;
import entity.Job;
import utility.MessageUI;
import java.util.InputMismatchException;
import java.util.Scanner;

public class JobManage {

    private final LinkedListInterface<Job> jobList;
    private final JobManagementUI ui;
    private final Scanner scanner = new Scanner(System.in);
    private final MessageUI messageUI = new MessageUI();


    public JobManage() {
        jobList = new LinkedList<>();
        ui = new JobManagementUI();
    }
```

```
    public void run() {
        jobList.add(new Job("J001", "Software Developer", "Develop and maintain Java applications.",
"TechSoft", "Kuala Lumpur", "Full-Time", 1500));
        jobList.add(new Job("J002", "App Developer", "Develop and maintain mobile applications.",
"MobileApp", "Penang", "Full-Time", 1000));
        jobList.add(new Job("J003", "Web Developer", "Develop and maintain websites.", "WebTech",
"Johor Bahru", "Full-Time", 1200));
        jobList.add(new Job("J004", "System Analyst", "Analyze and design systems for clients.",
"SysTech", "Kuala Lumpur", "Part-Time", 1300));
        jobList.add(new Job("J005", "Database Administrator", "Manage and maintain databases.",
"DBSolutions", "Penang", "Full-Time", 1600));

        boolean quit = false;
        while (!quit) {
            ui.jobManagementMenu();
            int choice = readInt();
            switch (choice) {
                case 1 -> listAllJobs();
                case 2 -> handleAddJob();
                case 3 -> handleUpdateJob();
                case 4 -> handleRemoveJob();
                case 5 -> handleFilterJob();
                case 6 -> generateJobReport();
                case 0 -> quit = true;
                default -> messageUI.displayInvalidChoiceMessage();
            }
        }
    }

    private int readInt() {
        while (true) {
            try {
                int i = scanner.nextInt();
                scanner.nextLine();
                return i;
            } catch (InputMismatchException e) {
                messageUI.displayInvalidChoiceMessage();
                scanner.nextLine();
            }
        }
    }

    public void addJob(Job job) {
        jobList.add(job);
        System.out.println("Job added successfully.");
    }

    public void listAllJobs() {
        if (jobList.isEmpty()) {
            messageUI.displayNoJobPostingsMessage();
        } else {
            for (int i = 1; i <= jobList.getNumberOfEntries(); i++) {
                ui.displayJob(jobList.getEntry(i), i);
            }
        }
    }
```

```
private void handleAddJob() {
    boolean addMore = true;
    while (addMore) {
        Job job = getJobDetails();
        addJob(job);
        System.out.print("Add another? (Y/N): ");
        String c = scanner.nextLine();
        if (!c.equalsIgnoreCase("Y")) {
            addMore = false;
        }
    }
}

private void handleUpdateJob() {
    System.out.print("Enter Job ID to update: ");
    String id = scanner.nextLine();
    Job job = findJobById(id);
    if (job == null) {
        messageUI.displayJobNotFoundMessage();
        return;
    }
    boolean editing = true;
    while (editing) {
        ui.updateOption();
        int opt = readInt();
        switch (opt) {
            case 1 -> {
                System.out.print("New Title: ");
                job.setTitle(scanner.nextLine());
            }
            case 2 -> {
                System.out.print("New Description: ");
                job.setDescription(scanner.nextLine());
            }
            case 3 -> {
                System.out.print("New Company: ");
                job.setCompanyName(scanner.nextLine());
            }
            case 4 -> {
                System.out.print("New Location: ");
                job.setLocation(scanner.nextLine());
            }
            case 5 -> {
                System.out.print("New Job Type: ");
                job.setJobType(scanner.nextLine());
            }
            case 6 -> {
                System.out.print("New Salary: RM ");
                if (scanner.hasNextDouble()) {
                    job.setSalary(scanner.nextDouble());
                    scanner.nextLine();
                } else {
                    messageUI.displayInvalidChoiceMessage();
                    scanner.nextLine();
                }
            }
            case 0 -> editing = false;
            default -> messageUI.displayInvalidChoiceMessage();
```

```
        }

    }
    messageUI.displayJobUpdatedMessage();
}
```

```
private void handleRemoveJob() {
    if (jobList.isEmpty()) {
        messageUI.displayNoJobPostingsMessage();
        return;
    }
    for (int i = 1; i <= jobList.getNumberOfEntries(); i++) {
        ui.displayJob(jobList.getEntry(i), i);
    }
    while (true) {
        System.out.print("Enter Job ID to remove (or 0 to cancel): ");
        String id = scanner.nextLine();
        if (id.equals("0")) {
            messageUI.displayRemoveCancelledMessage();
            return;
        }
        Job toRemove = findJobById(id);
        if (toRemove == null) {
            messageUI.displayJobIdNotFoundMessage();
        } else {
            System.out.print("Confirm remove \"" + id + "\"? (Y/N): ");
            String c = scanner.nextLine();
            if (c.equalsIgnoreCase("Y")) {
                for (int i = 1; i <= jobList.getNumberOfEntries(); i++) {
                    if (jobList.getEntry(i).getJobId().equalsIgnoreCase(id)) {
                        jobList.remove(i);
                        messageUI.displayRemovedJobMessage(toRemove);
                        updateJobIds();
                        return;
                    }
                }
            } else {
                messageUI.displayRemoveCancelledMessage();
                return;
            }
        }
    }
}

private void handleFilterJob() {
    ui.filterOption();
    int choice = readInt();
    switch (choice) {
        case 1 -> {
            System.out.print("Location (or 0 to cancel): ");
            filterByLocation(scanner.nextLine());
        }
        case 2 -> {
            System.out.print("Company  (or 0 to cancel): ");
            filterByCompany(scanner.nextLine());
        }
        case 3 -> {
            System.out.print("Job Type (or 0 to cancel): ");
            filterByType(scanner.nextLine());
        }
        case 4 -> {
            double min = readSalary("Min Salary");
            double max = readSalary("Max Salary");
            filterBySalary(min, max);
```

```
        }
        case 0 -> messageUI.displayFilterCancelledMessage();
    }
}

private Job findJobById(String id) {
    for (int i = 1; i <= jobList.getNumberOfEntries(); i++) {
        if (jobList.getEntry(i).getJobId().equalsIgnoreCase(id)) {
            return jobList.getEntry(i);
        }
    }
    return null;
}

private void filterByLocation(String loc) {
    if (loc.equals("0")) {
        messageUI.displayFilterCancelledMessage();
        return;
    }
    boolean found = false;
    for (int i = 1; i <= jobList.getNumberOfEntries(); i++) {
        Job j = jobList.getEntry(i);
        if (j.getLocation().equalsIgnoreCase(loc)) {
            ui.displayJob(j, i);
            found = true;
        }
    }
    if (!found) {
        messageUI.displayNoJobsFoundForFilter("location", loc);
    }
}

private void filterByCompany(String comp) {
    if (comp.equals("0")) {
        messageUI.displayFilterCancelledMessage();
        return;
    }
    boolean found = false;
    for (int i = 1; i <= jobList.getNumberOfEntries(); i++) {
        Job j = jobList.getEntry(i);
        if (j.getCompanyName().equalsIgnoreCase(comp)) {
            ui.displayJob(j, i);
            found = true;
        }
    }
    if (!found) {
        messageUI.displayNoJobsFoundForFilter("company", comp);
    }
}

private void filterByType(String type) {
    if (type.equals("0")) {
        messageUI.displayFilterCancelledMessage();
        return;
    }
    boolean found = false;
    for (int i = 1; i <= jobList.getNumberOfEntries(); i++) {
        Job j = jobList.getEntry(i);
```

```
        if (j.getJobType().equalsIgnoreCase(type)) {
            ui.displayJob(j, i);
            found = true;
        }
    }
    if (!found) {
        messageUI.displayNoJobsFoundForFilter("job type", type);
    }
}

private void filterBySalary(double min, double max) {
    boolean found = false;
    for (int i = 1; i <= jobList.getNumberOfEntries(); i++) {
        Job j = jobList.getEntry(i);
        if (j.getSalary() >= min && j.getSalary() <= max) {
            ui.displayJob(j, i);
            found = true;
        }
    }
    if (!found) {
        messageUI.displayNoJobsFoundForFilter("salary range", "RM " + min + " - RM " + max);
    }
}

private Job getJobDetails() {
    String id = generateJobId();
    System.out.println("Generated Job ID: " + id);
    System.out.print("Title: ");
    String t = scanner.nextLine();
    System.out.print("Description: ");
    String d = scanner.nextLine();
    System.out.print("Company: ");
    String c = scanner.nextLine();
    System.out.print("Location: ");
    String l = scanner.nextLine();
    System.out.print("Job Type: ");
    String jt = scanner.nextLine();
    double s = readSalary("Salary");
    return new Job(id, t, d, c, l, jt, s);
}

private double readSalary(String p) {
    while (true) {
        System.out.print(p + " (RM): ");
        try {
            double v = scanner.nextDouble();
            scanner.nextLine();
            return v;
        } catch (InputMismatchException e) {
            messageUI.displayInvalidChoiceMessage();
            scanner.nextLine();
        }
    }
}
```

```
private String generateJobId() {
    return String.format("J%03d", jobList.getNumberOfEntries() + 1);
  }

  private void updateJobIds() {
    for (int i = 1; i <= jobList.getNumberOfEntries(); i++) {
      jobList.getEntry(i).setJobId(String.format("J%03d", i));
    }
  }

  public void generateJobReport() {
    System.out.println("\n|                    Job Summary Report                    |");
    System.out.println("-----------------------------------------------------------------------------------");
    System.out.println("| Job ID | Title              | Company       | Type      | Salary    |");
    System.out.println("-----------------------------------------------------------------------------------");
    for (int i = 1; i <= jobList.getNumberOfEntries(); i++) {
      Job j = jobList.getEntry(i);
      System.out.printf("| %-6s | %-20s | %-13s | %-9s | RM %-7.2f |\n",
          j.getJobId(), j.getTitle(), j.getCompanyName(), j.getJobType(), j.getSalary());
    }
    System.out.println("-----------------------------------------------------------------------------------");
  }

  public LinkedListInterface<Job> getAllJobs(){
    return  jobList;
  }

}
```

2. Screenshots

```
=========== Internship Applicaiton ===========
1. Job Management
2. Applicant Management
3. Search
0. Exit system
==========================================
Enter Choice: 1
```

When the application starts, the system will display the main menu, then we enter 1 to go to the Job Management module.

```
=========== Job Management ===========
1. List all jobs
2. Add new job posting
3. Update existing job posting
4. Remove job posting
5. Filter job posting
6. Generate job report
0. Back to Main Menu
==========================================
Enter Choice: 1
```

```
================ Job 1 ================
ID          : J001
Title       : Software Developer
Description : Develop and maintain Java applications.
Company     : TechSoft
Location    : Kuala Lumpur
Type        : Full-Time
Salary      : RM 1500.00
=======================================

================ Job 2 ================
ID          : J002
Title       : App Developer
Description : Develop and maintain mobile applications.
Company     : MobileApp
Location    : Penang
Type        : Full-Time
Salary      : RM 1000.00
=======================================

================ Job 3 ================
ID          : J003
Title       : Web Developer
Description : Develop and maintain websites.
Company     : WebTech
Location    : Johor Bahru
Type        : Full-Time
Salary      : RM 1200.00
=======================================
```

```
================ Job 4 ================
ID          : J004
Title       : System Analyst
Description : Analyze and design systems for clients.
Company     : SysTech
Location    : Kuala Lumpur
Type        : Part-Time
Salary      : RM 1300.00
=======================================

================ Job 5 ================
ID          : J005
Title       : Database Administrator
Description : Manage and maintain databases.
Company     : DBSolutions
Location    : Penang
Type        : Full-Time
Salary      : RM 1600.00
=======================================
```

After entering the Job Management module, the system displays the Job Management menu, then we enter 1 to list all the existing jobs.

```
=========== Job Management ===========
1. List all jobs
2. Add new job posting
3. Update existing job posting
4. Remove job posting
5. Filter job posting
6. Generate job report
0. Back to Main Menu
========================================
Enter Choice: 2
Generated Job ID: J006
Title: Software Engineer
Description: Develop software
Company: HighTech
Location: Kuala Lumpur
Job Type: Full-Time
Salary (RM): 1500
Job added successfully.
Add another? (Y/N): n
```

After that, we enter 2 to add a new job posting, then the system will auto generate the job ID and require user input the job detail. After finishing inputting the details, the system displays the job added successfully message and asks the user whether they want to add another or not, then we enter no, if enter yes the system will create the new job posting and ask user input for the new job posting.

```
=========== Job Management ===========
1. List all jobs
2. Add new job posting
3. Update existing job posting
4. Remove job posting
5. Filter job posting
6. Generate job report
0. Back to Main Menu
========================================
Enter Choice: 3
Enter Job ID to update: J006
================ Update ================
Select what you want to update:
1. Title
2. Description
3. Company Name
4. Location
5. Job Type
6. Salary
0. Cancel
========================================
Enter choice: 5
New Job Type: Internship
================ Update ================
Select what you want to update:
1. Title
2. Description
3. Company Name
4. Location
5. Job Type
6. Salary
0. Cancel
========================================
Enter choice: 0
Job updated.
```

After finishing the added process, we select 3 to update the existing job posting. Then the system displays the update menu for the user to select what field the user wants to update. Now we select 5 which is to update the job type to internship, then the system will display the menu for users to continue to select what they want to update, then we select 0 to exit the update process then the system also displays the job updated message which tells the user the update is successful.

```
=========== Job Management ===========
1. List all jobs
2. Add new job posting
3. Update existing job posting
4. Remove job posting
5. Filter job posting
6. Generate job report
0. Back to Main Menu
========================================
Enter Choice: 4

================ Job 1 ================
ID          : J001
Title       : Software Developer
Description : Develop and maintain Java applications.
Company     : TechSoft
Location    : Kuala Lumpur
Type        : Full-Time
Salary      : RM 1500.00
========================================

================ Job 2 ================
ID          : J002
Title       : App Developer
Description : Develop and maintain mobile applications.
Company     : MobileApp
Location    : Penang
Type        : Full-Time
Salary      : RM 1000.00
========================================
```

```
================ Job 3 ================
ID          : J003
Title       : Web Developer
Description : Develop and maintain websites.
Company     : WebTech
Location    : Johor Bahru
Type        : Full-Time
Salary      : RM 1200.00
========================================

================ Job 4 ================
ID          : J004
Title       : System Analyst
Description : Analyze and design systems for clients.
Company     : SysTech
Location    : Kuala Lumpur
Type        : Part-Time
Salary      : RM 1300.00
========================================

================ Job 5 ================
ID          : J005
Title       : Database Administrator
Description : Manage and maintain databases.
Company     : DBSolutions
Location    : Penang
Type        : Full-Time
Salary      : RM 1600.00
========================================

================ Job 6 ================
ID          : J006
Title       : Software Engineer
Description : Develop software
Company     : HighTech
Location    : Kuala Lumpur
Type        : Internship
Salary      : RM 1500.00
========================================
```

```
Enter Job ID to remove (or 0 to cancel): J005
Confirm remove "J005"? (Y/N): y
Removed: J005 | Database Administrator | DBSolutions
```

Next we select 4 to remove job posting, the system will display the existing job including the new job that we added just now. Then we input “J005” to remove it, the system will ask a confirmation whether confirm remove the J005 job post. We enter “y” to confirm removal then the system displays the job that we remove.

```
=========== Job Management ===========
1. List all jobs
2. Add new job posting
3. Update existing job posting
4. Remove job posting
5. Filter job posting
6. Generate job report
0. Back to Main Menu
========================================
Enter Choice: 5

================ Filter ================
1. Location
2. Company
3. Job Type
4. Salary Range
0. Cancel
========================================
Enter choice: 3
Job Type (or 0 to cancel): Internship

================ Job 5 ================
ID          : J005
Title       : Software Engineer
Description : Develop software
Company     : HighTech
Location    : Kuala Lumpur
Type        : Internship
Salary      : RM 1500.00
========================================
```

After we finish removing jobs, we select 5 for filter job posting. Then the system displays a filter menu for user select want filter based on what criteria. Here we input “3” which is job type, then we input “Internship” which we just added, then the system will display the job posting.

```
=========== Job Management ===========
1. List all jobs
2. Add new job posting
3. Update existing job posting
4. Remove job posting
5. Filter job posting
6. Generate job report
0. Back to Main Menu
========================================
Enter Choice: 6

|                                Job Summary Report                                |
-------------------------------------------------------------------------------------
| Job ID | Title                  | Company        | Type       | Salary     |
-------------------------------------------------------------------------------------
| J001   | Software Developer     | TechSoft       | Full-Time  | RM 1500.00 |
| J002   | App Developer          | MobileApp      | Full-Time  | RM 1000.00 |
| J003   | Web Developer          | WebTech        | Full-Time  | RM 1200.00 |
| J004   | System Analyst         | SysTech        | Part-Time  | RM 1300.00 |
| J005   | Software Engineer      | HighTech       | Internship | RM 1500.00 |
-------------------------------------------------------------------------------------
```

At the end, we input “6” to generate the job report, then the system will display all the jobs that exist in the system.